A Lightweight Hardware-Counter Phase Detector for Multicore Workloads

Keshav Krishna

Abstract

Efficiency-oriented processors increasingly depend on workload-aware hardware adaptation: static choices for frequency, cache policy, prefetching, and scheduling leave performance and energy opportunities unused when software behavior changes over time. Prior work has shown that programs exhibit recurring phases and that phase information can guide simulation, resource management, and forecasting. However, many established phase-detection techniques are awkward for direct online hardware use: offline SimPoint-style clustering requires profile construction, runtime clustering can be noisy and threshold-sensitive, and neural phase-aware predictors add storage and inference complexity. This paper studies a more hardware-practical path for multicore workloads: train-only offline clustering over the full safe hardware-counter set to define phase labels, counter-family ablations to select one representative counter per family, and shallow decision trees that use the last 20 selected-counter intervals to predict the next 5 phase labels. The goal is not to maximize model sophistication, but to distill a richer offline phase teacher into a detector that is implementable with a small set of monitored counters, bounded history storage, and threshold-comparator logic. On PARSEC-derived multicore traces collected under three workload-construction regimes, the reduced five-counter detector achieves mean top-1 accuracies of 75.0%, 83.5%, and 65.0% for Sets 1–3, respectively, while maintaining transition-case accuracies of 28.1%, 47.4%, and 32.1%. Against simple online baselines, the detector improves mean accuracy over last-state persistence by 2.6, 5.6, and 1.6 percentage points, respectively, while preserving a compact depth-6 implementation. The resulting five-tree horizon predictors require an estimated 1.26–1.77 KB of model storage per set-level detector, showing that offline clustering can be converted into a low-latency online phase forecaster rather than deployed directly in hardware.

1 Introduction

Modern processors are built to be configurable. Voltage/frequency settings, cache and prefetch controls, thread placement, and power-management policies expose many ways to trade performance, energy, latency, and thermal headroom. The hard part is deciding when to change those controls. A static configuration chosen at boot time, process launch, or benchmark start cannot track the fact that a multicore workload may alternate between compute-heavy, memory-bound, branch-sensitive, and synchronization-heavy intervals.

Program phase behavior provides the missing abstraction. If intervals with similar execution behavior can be recognized, hardware or the operating system can avoid reacting to every raw counter fluctuation and instead make decisions over a compact phase state. Offline phase analysis has been effective for simulation-point selection using Basic Block Vectors (BBVs), SimPoint, and clustering [35, 36, 14]. Runtime phase trackers showed that compact fingerprints and Markov predictors can expose phase transitions online [37, 7]. Parallel phase studies extended the idea to per-thread and shared-memory behavior, where phase identity interacts with scaling and shared-resource interference [31, 12]. More recent work uses hardware performance counters (HPCs), clustering, regression, and neural models to connect phases with power, cache demand, scheduling, and forecasting [21, 39, 30, 26, 6].

These systems also expose a tension. The most accurate phase representation is not necessarily the best representation for hardware control. BBV-based approaches are powerful but usually rely on offline profiling or code signatures. Online clustering and GMM variants must maintain centroids or distributions and can create spurious short phases unless filtered. Neural and LSTM-based predictors improve sequence modeling, but inference latency, parameter storage, quantization, and integration into low-level control loops are nontrivial. Finally, many PMU-based methods begin with broad event sets, even though real processors expose a limited number of programmable counters at once and many events are correlated.

This paper therefore asks a deliberately constrained question: how far can we get with a small set of counters and a decision tree? The proposed flow first reduces redundancy using correlation analysis, excludes timing-dependent features that would leak DVFS decisions back into the detector, runs counter-family ablations to select representative events, defines three offline phase labels by clustering the full safe counter vector, and then trains a compact bank of decision trees over short selected-counter histories. The online detector emits a five-step phase forecast over low-, moderate-, and high-pressure states, so a controller can reevaluate every interval or amortize control decisions across several future intervals.

This paper makes the following contributions:

- A correlation-driven counter-reduction workflow that identifies redundant PMU indicators before model training.
- A counter-family ablation methodology that evaluates cache, memory/off-core, branch-control, and

instruction/floating-point families separately.

- A dataset collection and merging pipeline for three multicore workload regimes: single process with multiple threads, multiple single-threaded processes, and hybrid multiprocess/multithread workloads.
- A phase-detector distillation flow in which train-only offline k-means over the full safe counter set defines three interpretable phase labels, while the deployed detector uses only one counter per family.
- A long-horizon online detector that uses the last 20 intervals to predict the next 5 phases with shallow decision trees.
- A hardware-overhead analysis showing how the resulting tree bank maps to comparators, small tables, and bounded per-core history state, plus an evaluation against persistence and majority baselines.

2 Related Work

The strongest early evidence for phase behavior came from code-signature methods for simulation and architectural characterization. Basic Block Distribution Analysis and SimPoint showed that normalized basic-block vectors can expose recurring behavior and select representative intervals for simulation [35, 36]. Later SimPoint work improved scalability and flexibility, while phase-classification studies compared alternative code structures and detection metrics such as basic blocks, procedures, opcodes, loop branches, instruction working sets, and conditional branches [14, 22, 11]. This line of work is essential because it made phase behavior measurable, but its usual data path is profiling-oriented and code-centric rather than a direct implementation target for hardware control loops.

Runtime phase work moved the idea closer to online adaptation. Working-set signatures were used to manage multiconfiguration hardware and avoid repeated retuning [10], while runtime phase tracking used compact instruction fingerprints and predictors to classify and forecast large-scale behavior [37]. Sampling-based approaches extended this direction to multithreaded execution and showed the value of classifying intervals without observing every instruction [7]. These systems motivate our use of history and hysteresis, but they still depend on code signatures or tuning structures that differ from the threshold-comparator datapath considered here.

Parallel and multicore phase behavior adds another layer of difficulty. Per-thread representations are needed when the same application changes behavior across threads or thread counts [31], and broader phase-exploitation work argues that phases can appear at multiple granularities in memory reference behavior, I/O activity, and resource occupancy [12]. Multilevel phase analysis further shows that phases can be nested and tied to different hardware resources [38]. The PARSEC suite is a useful substrate for this question because it was designed to stress shared-memory multicore behavior, locality, synchronization, and off-chip traffic [3]. These observations are why this paper separates single-process multithreaded, multiprocess single-threaded, and hybrid workloads instead of assuming that one aggregate process signal is sufficient.

Hardware performance counters provide a different interface to phase behavior. Counter architectures and vendor manuals describe both the opportunity and the ambiguity of PMU measurements: counters are cheap relative to instrumentation, but event meanings, availability, multiplexing, and overlap effects require care [13, 17, 18, 23, 24]. Nomani and Szefer use counters for phase prediction in a security-aware scheduling context [30]; Ziedan et al. use PMU-derived cache-access signatures for runtime L2 cache-demand phases [39]; Kim et al. use phase information and neural models for cross-platform power/performance prediction [21]; and Lozano and Gerstlauer combine clustering, classification, prediction, and LSTMs for phase-aware multicore workload forecasting [26]. These works support the use of counters and learning, but they also expose the cost of broad counter sets, offline tuning, centroid maintenance, or neural inference in a low-level controller.

The control targets for phase detection are well established. Adaptive cache ways, reconfigurable memory hierarchies, and adaptive issue queues show that hardware structures can save energy or improve performance when their active resources track workload demand [1, 2, 5]. Power-monitoring and scheduling systems use counters or workload characterization to guide thermal balancing, virtual-machine management, heterogeneous-core scheduling, and frequency decisions [19, 29, 9, 32, 6]. Low-level implementations also need deterministic state updates and compact policy tables; techniques such as read-copy update illustrate the broader systems concern of updating shared control state without long stalls [28]. Our detector is not itself a cache, DVFS, or scheduler policy, but is intended to supply the stable state signal those mechanisms need.

The modeling tools used by prior phase systems span classic clustering, probabilistic models, decision trees, and neural sequence models. K-means and related clustering methods are a natural fit for SimPoint-style interval grouping [27, 25, 20]; EM underlies Gaussian-mixture approaches [8]; CART and C4.5 motivate the decision-tree representation used here [4, 33]; and back-propagation and LSTM models motivate neural baselines for sequence prediction [34, 16]. General architecture texts frame the hardware tradeoff behind this choice: predictor accuracy matters, but storage, latency, counters, and control-loop stability matter too [15].

This work is closest in spirit to online phase tracking and PMU-driven phase prediction, but differs in two ways. First, it explicitly minimizes the hardware-visible counter set before classifier training using correlation analysis and family ablations. Second, it replaces online clustering or neural inference with a compact decision-tree classifier whose internal nodes can be implemented as threshold comparators and whose output can feed a small phase-to-action table.

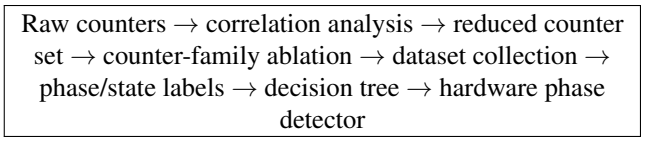


Figure 1: Phase-detector pipeline. Each arrow is a data-reduction or representation step intended to make the final detector easier to map into hardware.

3 Methodology

3.1 Overview

Figure 1 summarizes the pipeline. Raw PMU measurements are first analyzed for redundancy. Timing-dependent counters and derived features are excluded from the phase-state modeling path. The remaining counters are grouped into semantic families, ablated to select representative signals, collected again in set-specific multicore experiments, transformed into three-state sequences, and used to train a decision tree over a short history window.

3.2 Initial Counter Set

The initial counter set is derived from Linux `perf` / PMU event availability and the repository's event-aliasing logic. The correlation artifact reports ten confident core PMU counters: instructions retired, cycles, branch instructions, branch mispredictions, L1 data loads, L1 data stores, LLC references, LLC misses, off-core demand data reads, and resource stalls. The same artifact also records system-wide uncore bandwidth columns when host policy permits them: memory read bandwidth, memory write bandwidth, and total memory bandwidth. On the Xeon E5-2680 v2 system used for the correlation artifact, these map to preferred Intel events such as `inst_retired.any`, `cpu_clk_unhalted.thread`, `br_inst_retired.all_branches`, `br_misp_retired.all_branches`, `mem_uops_retired.all_loads`, `mem_uops_retired.all_stores`, `longest_lat_cache.reference`, `longest_lat_cache.miss`, `offcore_requests.demand_data_rd`, and `resource_stalls.any`.

3.3 Excluding Cycle/Timing-Dependent Counters

The detector should not learn a phase state that changes simply because the controller changed frequency. Accordingly, the modeling path excludes counters or features whose names indicate timing-derived behavior, including cycle, IPC, CPI, elapsed time, stalls, per-ms rates, interval duration, and runtime. This policy follows the implementation in `phase_family_ml/families.py`. The correlation study may still report timing-derived indicators, because they are useful for understanding redundancy and workload behavior, but the hardware phase detector should avoid using them as direct inputs.

3.4 Correlation-Guided Counter Reduction

The correlation analysis in `results_indep_study/` uses raw and normalized views. The normalized Spearman view is the preferred basis for redundancy decisions because raw counts can correlate merely because total activity, interval length, or thread count increased. The current recommendation table identifies two high-correlation groups: cycles/ms with instructions retired/ms, and LLC references/KI with off-core demand reads/KI, both with maximum $|\rho| = 0.976$. For the final detector, this step is used to avoid monitoring counters that convey near-duplicate signal under the normalized view.

3.5 Counter-Family Ablation

Counters are grouped by semantic family: L1, L2, LLC, memory/off-core, branch-control, and core instruction/floating-point behavior. The ablation procedure evaluates singleton counters within each family and an exhaustive one-per-family combination when coverage permits. In the current large set artifacts, L2 is unavailable because no L2 counter sequence rows are present. The selected one-per-family sets differ across workload regimes, which is itself a useful warning: counter reduction should be evaluated under the intended deployment mix rather than assumed universal.

3.6 Dataset Collection and Merging

The collection pipeline records PARSEC interval traces, aligns raw `perf` output with workload metadata, merges raw run folders into interval datasets, and preserves metadata such as workload, thread count, run identifier, core identifier, and phase label. Table 1 summarizes the current large artifacts.

3.7 Phase/State Definition

The current pipeline maps each interval into three discrete phase states:

$$s_t \in \{0, 1, 2\} = \{\text{low}, \text{moderate}, \text{high}\}. \tag{1}$$

The state definition is intentionally coarse. For dynamic reconfiguration, the detector only needs to distinguish whether resource pressure is low enough to reduce provisioning, high enough to increase provisioning, or in the middle where the current policy may be retained. We use offline clustering over the full safe counter set only to define phase labels. The deployed detector does not perform clustering; instead, it uses a shallow decision tree trained to approximate those labels from a reduced counter set, making the phase detector suitable for low-latency hardware implementation. The sequence builder fits the scaler and k-means centroids only on training intervals, assigns validation and test intervals by nearest centroid, and orders the three clusters by a resource-pressure score based on

Table 1: Current dataset artifacts. Counts are sourced from the local merge summaries.

Set	Regime	Runs	Rows	Uncore rows
Set 1	single-process multithreaded	1050	685787	466743
Set 2	multiprocess single-threaded	240	25627	17720
Set 3	hybrid multiprocess/multithread	240	90876	67980

Algorithm 1 Multi-step decision-tree phase detector

Require: Selected per-core counter vector c_t, history window W=20, trained tree bank $\{T_1, \ldots, T_5\}$
Ensure: Forecast $\hat{s}_{t+1:t+5} \in \{\text{low}, \text{moderate}, \text{high}\}^5$
1: Normalize or quantize each selected counter in c_t
2: Append the quantized vector to the per-core history buffer
3: Construct window features x_t from the most recent W intervals
4: **for** $h \in \{1, \ldots, 5\}$ **do**
5: Traverse T_h by comparing feature values against node thresholds
6: Let $\hat{s}_{t+h}$ be the phase state stored in the reached leaf
7: **end for**
8: Publish the horizon forecast or derive a control action from it

LLC misses, memory bandwidth, branch mispredictions, and stalls when available. After ablation selects one representative raw counter per family, the selected-counter streams retain the same clustered phase labels. This ordering step is important for interpretability: k-means itself produces anonymous clusters, while the pressure sort turns them into a low/moderate/high resource-pressure axis that can be checked directly in the generated cluster summaries.

3.8 Decision-Tree Training over Counter Sequences

The phase-detector command in the repository trains a bank of five shallow decision trees. Each tree consumes a flattened history of the last 20 selected-counter vectors and predicts one future clustered phase label, from t+1 through t+5. The intended hardware detector uses the same basic structure: normalize or quantize the selected counters, maintain a bounded history buffer, traverse a small tree, and emit a future phase state. Because the predictor is multi-step rather than single-step, hardware control logic can invoke the detector every interval or less frequently and use the predicted horizon between invocations.

3.9 Stability Filtering / Hysteresis

A phase detector used for hardware control must avoid oscillation. The proposed detector therefore separates label prediction from action publication. The current final-dataset artifacts report stable-case accuracy and transition-case accuracy directly from the raw five-step predictions; a downstream controller can still apply hysteresis or action smoothing on top of those predictions. This mirrors the role of confidence tables and transition phases in prior online phase trackers, but keeps the detector state minimal: a bounded history buffer, a tree bank, and optional action-side hysteresis.

3.10 Output Actions for Hardware Reconfiguration

The detector does not prescribe one fixed hardware action. Instead, it provides a compact state that can index a phase-to-action table for DVFS, cache partitioning or way control, prefetch aggressiveness, scheduling, or power gating. For example, a high memory/off-core state may request a different prefetch or cache policy, while a low instruction/floating-point state may permit lower frequency. These action mappings must be evaluated separately from detector accuracy. The evaluation therefore distinguishes measured detector overhead from projected system-level energy or performance savings.

4 Experiments

4.1 Research Questions

- RQ1: Which counters are redundant and can be removed without losing phase signal?
- RQ2: Which counter family is most predictive of phase behavior under each workload regime?
- RQ3: Do the clustered three-state labels correspond to interpretable low-, moderate-, and high-pressure workload states?
- RQ4: How well does the reduced-counter long-horizon detector reproduce those labels, and how does it compare with simple online baselines?
- RQ5: What are the detector's latency, storage, and hardware overhead?

4.2 Experimental Setup

The correlation artifact reports an Intel Xeon E5-2680 v2 host with 20 logical CPUs, two sockets, 10 cores per socket, Linux `perf` version 5.14.0, PARSEC availability, and 10 ms target sampling intervals. The benchmark suite is PARSEC [3]. The final collection artifacts under `/scratch/kk6081/finals_dataset/set1`, `/scratch/kk6081/finals_dataset/set2`, and `/scratch/kk6081/finals_dataset/set3` include

PARSEC workloads blackscholes, bodytrack, canneal, fluidanimate, freqmine, swaptions, and streamcluster across `simsmall`, `simmedium`, and `simlarge` inputs. The phase-family configuration uses history length 20, prediction horizon 5, random seed 17, a 70/15/15 train/validation/test split, and `config_group_holdout` grouping so that repetitions of the same collection configuration do not cross split boundaries. Offline k-means is fit only on training rows, and validation/test rows are labeled by nearest-centroid assignment from the trained clustering model.

4.3 Experiment Sets

Set 1 studies a single process with multiple threads. Set 2 studies multiple single-threaded processes running concurrently. Set 3 studies a hybrid construction with multiple processes and multiple threads. These sets are intended to separate intra-application parallelism from multiprogram interference and from the mixed case that combines both.

4.4 Baselines

The generated artifacts support three direct baselines for every set. The *last-state* baseline predicts that the future phase equals the current phase. The *majority* baseline predicts the globally most common future phase from the training rows. The *state-conditioned majority* baseline predicts the most common future phase conditioned on the current state. These baselines are intentionally simple: they separate persistence effects from genuine forecasting value without introducing a second expensive teacher model into the online comparison.

4.5 Metrics

The main classifier metrics are accuracy (equivalently top-1 accuracy), macro F1, recall of the high-pressure state, stable-case accuracy, and transition-case accuracy. Stable-case accuracy measures rows where the future label equals the current label; transition-case accuracy measures rows where the future label changes. This split matters for hardware use: persistence-heavy tasks can look strong under raw accuracy even when the detector fails to anticipate changes. We also report gains over the three online baselines, history length, prediction horizon, feature count, tree depth, node counts, and estimated storage. Energy/performance improvement is not reported because no DVFS, cache, or scheduling action policy is applied in these artifacts.

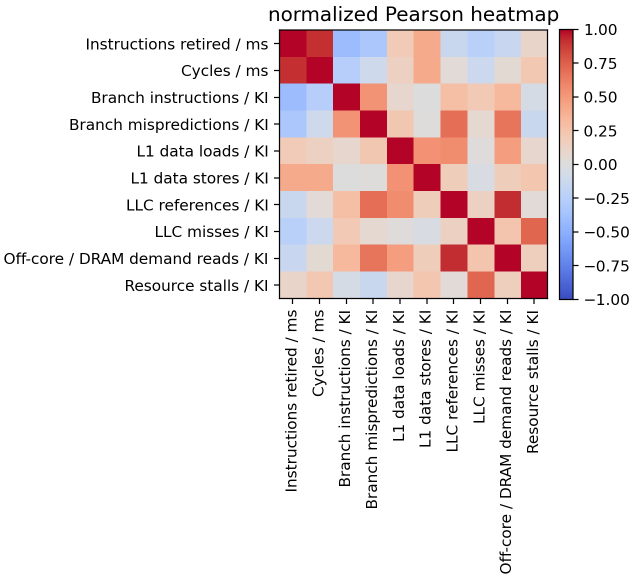


Figure 2: Normalized Pearson correlation heatmap from the correlation study.

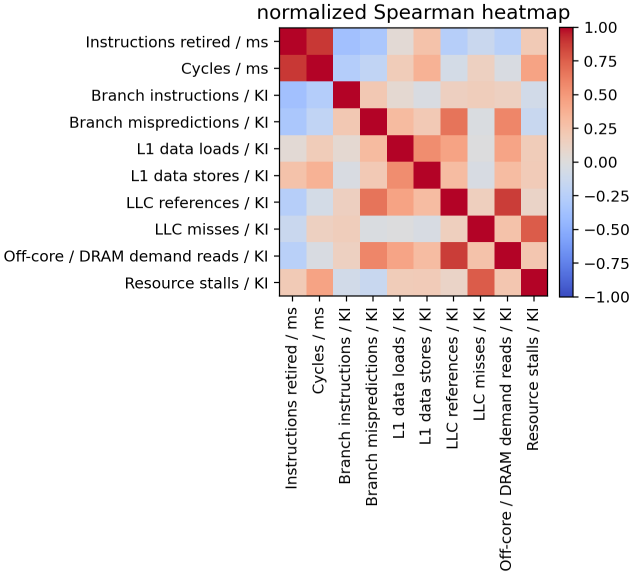


Figure 3: Normalized Spearman correlation heatmap. This view is the primary basis for redundancy decisions.

5 Results

5.1 Correlation Structure and Counter Redundancy

The normalized correlation artifact contains 9306 analysis-ready rows after dropping startup or invalid rows from 9409 interval rows. It reports two reduction recommendations at $|\rho| = 0.976$: keep cycles/ms over instructions retired/ms in the timing-normalized view, and keep LLC references/KI over off-core demand reads/KI. Because timing-derived counters are excluded from the detector, the second group is the more directly relevant detector input reduction.

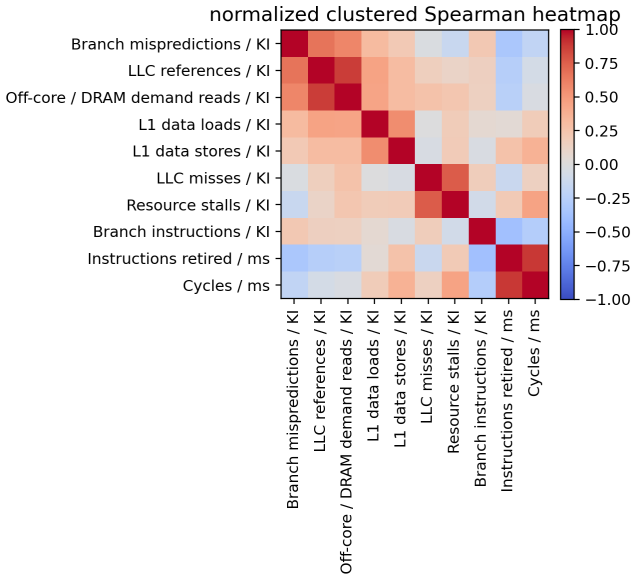


Figure 4: Clustered normalized correlation heatmap.

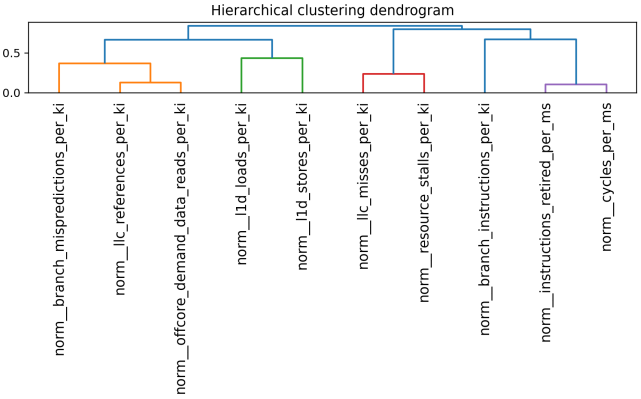


Figure 5: Correlation dendrogram used to identify redundant indicator groups.

5.2 Selected and Removed Counters

Table 2 reports the selected one-per-family sets from the ablation artifacts. The selections are not identical across sets: Set 2 alone prefers LLC misses, while Set 3 alone prefers L1 stores and read bandwidth. This supports evaluating counter reduction under multiple workload-construction regimes instead of assuming a single universal event set. Figure 8 shows the singleton counter scores that led to those choices; selected counters are shown in green.

5.3 Interpretability of Clustered States

Table 3 summarizes the pressure ordering learned by the offline clustering stage. In all three sets, state 2 is clearly the highest-pressure cluster after ordering by LLC misses, memory

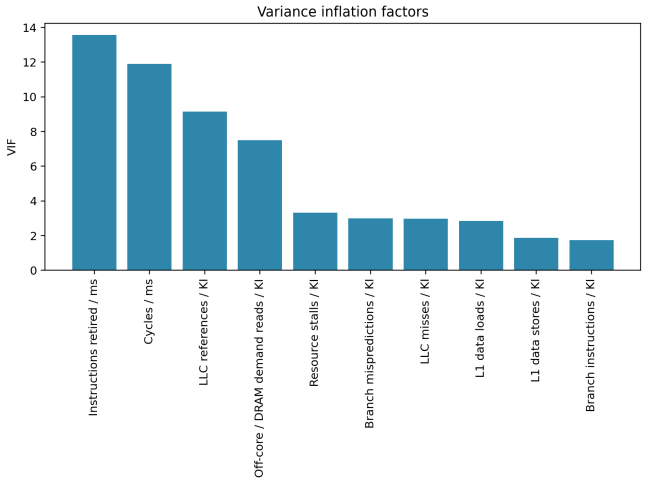


Figure 6: Variance inflation factors for normalized indicators.

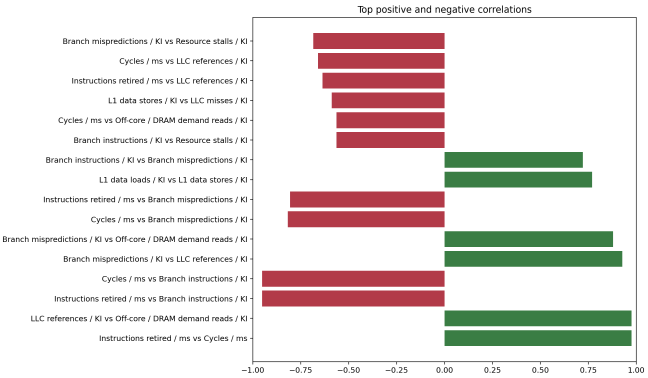


Figure 7: Top positive and negative normalized correlations.

bandwidth, branch mispredictions, and stalls when present. The strongest evidence is the pressure gap to state 2: Set 1 separates state 2 from state 1 by roughly 7.85 normalized pressure units, Set 2 by 7.95 units, and Set 3 by 3.32 units. This supports the claim that the labels are not arbitrary buckets, but recurring workload behaviors with a consistent high-pressure endpoint.

The low and moderate states are less sharply separated, especially in Set 2 where the ordered pressure scores are close (-1.46 versus -1.21). That is a useful caution for the paper's framing: three-state clustering does produce a defensible low/moderate/high ordering, but the moderate state is best interpreted as a mixed transition region rather than a perfectly isolated operating mode.

5.4 Long-Horizon Detector Fidelity

Table 4 reports the final global phase-detector artifacts. Each set uses five shallow depth-6 trees trained on 100 history features (five selected counters across 20 intervals). The strongest mean accuracy appears in Set 2 at 83.5%, followed by Set 1 at 75.0% and Set 3 at 65.0%. Accuracy declines from $t{+}1$ to $t{+}5$ in every set, but the degradation is modest in Set 2 (85.0% to 83.1%) and larger in Set 3 (68.9% to 62.2%).

Table 2: Selected one-per-family counter sets from local ablation artifacts. L2 is unavailable in the current sequence artifacts.

Set	L1	LLC	Branch/control	Core/FP	Memory/off-core
Set 1	l1d_loads	llc_references	branch_instructions	instructions_retired	total_memory_bandwidth
Set 2	l1d_loads	llc_misses	branch_instructions	instructions_retired	total_memory_bandwidth
Set 3	l1d_stores	llc_references	branch_instructions	instructions_retired	memory_read_bandwidth

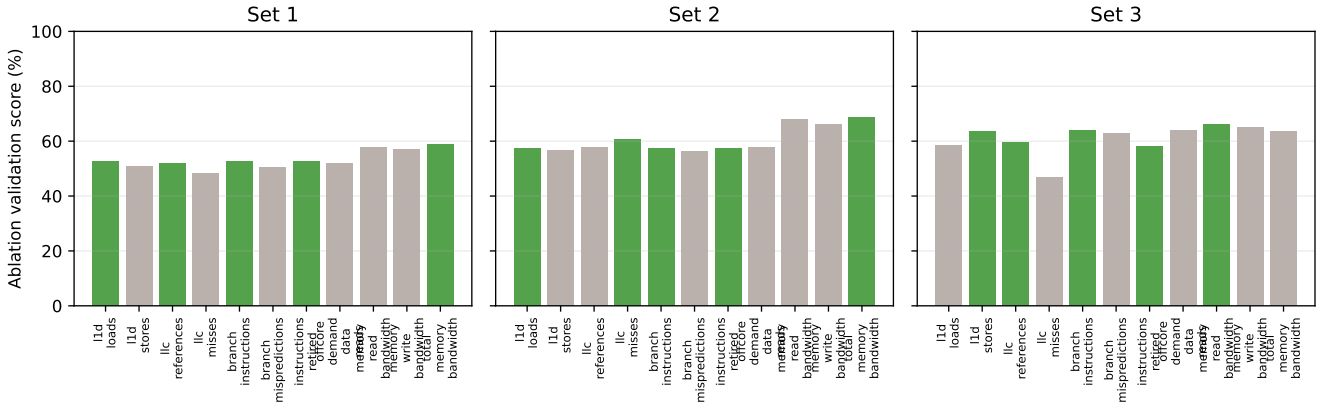


Figure 8: Singleton counter-family ablation scores. Green bars are the selected representative counters; gray bars are alternatives that were removed from the reduced detector input set.

Macro F1 is lower than accuracy in all sets because the ordered phase labels are imbalanced. Set 2 is the clearest example: raw accuracy is high because the dominant phase is very persistent, but mean macro F1 is only 47.7%. This is exactly why the paper should report stable- and transition-case behavior in addition to aggregate accuracy.

5.5 Accuracy versus Number of Counters

Figure 9 plots validation score against the number of counters in the ablation candidate. Singleton candidates expose the best event inside each family; exhaustive one-per-family candidates combine five monitored counters because L2 is absent in the current sequence artifacts. The selected five-counter combinations score 53.7%, 59.7%, and 63.7% validation score for Sets 1–3, respectively, in the ablation stage. This result is not an all-counter comparison; rather, it shows that the ablation search can choose a compact five-counter detector input while retaining useful label fidelity before the long-horizon detector is trained.

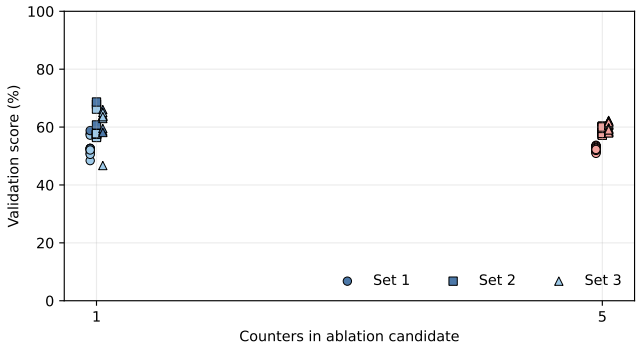


Figure 9: Ablation validation score versus number of counters in the candidate set. Points at one counter are singleton family candidates; points at five counters are one-per-family combinations.

5.6 Decision Tree versus Simple Baselines

Figure 10 provides a compact comparison view for the detector and the simple online baselines. The majority baseline confirms that the task is not solved by always predicting the dominant phase: its mean macro F1 is 26.8%, 45.4%, and 23.5% for Sets 1–3. The last-state baseline is much stronger because adjacent intervals are often persistent, reaching mean accuracies of 72.3%, 77.9%, and 63.4%. The reduced-counter tree improves mean accuracy over last-state in all three sets by 2.6, 5.6, and 1.6 percentage points, respectively.

The more important result is on transitions. Last-state prediction has zero transition-case accuracy by construction, while the tree reaches 28.1%, 47.4%, and 32.1% mean transition-case accuracy in Sets 1–3. The state-conditioned majority baseline captures some coarse label bias, but still trails the tree on both accuracy and transition sensitivity. These numbers support the detector's main value proposition: it is not just a persistence heuristic, but a cheap student of a richer offline phase teacher.

Table 3: Interpretability summary for train-only clustered phase labels. Pressure scores are the ordered training-set means of the normalized pressure proxy used to rename the clusters. Cluster shares use all labeled rows.

Set	Ordered pressure scores (state 0 / 1 / 2)	Cluster shares (state 0 / 1 / 2)
Set 1	$-2.40/0.13/7.97$	$28.6\%/63.7\%/7.7\%$
Set 2	$-1.46/-1.21/6.74$	$12.0\%/71.1\%/16.9\%$
Set 3	$-1.92/-1.30/2.01$	$44.3\%/7.2\%/48.5\%$

Table 4: Final long-horizon global detector results. Values are percentages except storage.

Set	$t{+}1$ acc.	$t{+}5$ acc.	Mean acc.	Mean macro F1	Mean transition acc.	Storage (B)
Set 1	77.9	73.4	75.0	51.4	28.1	1773
Set 2	85.0	83.1	83.5	47.7	47.4	1259
Set 3	68.9	62.2	65.0	44.8	32.1	1509

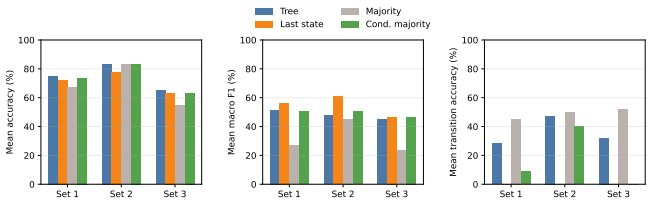


Figure 10: Baseline comparison view. The final detector should be interpreted using accuracy, macro F1, and stable/transition-case accuracy together rather than macro F1 alone.

5.7 Tree Depth and Hardware Cost

All final detectors use depth-6 trees, so a straightforward serial traversal requires at most six threshold comparisons per tree. Because the horizon predictor uses five trees, the relevant hardware question is aggregate storage across the tree bank rather than storage of a single family tree. The generated manifests report total storage estimates of 1773 B, 1259 B, and 1509 B for Sets 1–3. These correspond to 315/320, 234/239, and 268/273 internal-node/leaf counts across the five-tree bank, respectively. The feature vector is fixed at 100 history features in all sets, coming from five selected counters over 20 intervals.

6 Hardware Overhead

6.1 Mapping a Tree to Hardware

A decision tree has a direct hardware interpretation. Each internal node stores a feature identifier, a threshold, and child pointers. At inference time, the detector compares one quantized feature against the threshold and selects the next node. Each leaf stores a phase identifier. Traversal can be implemented as a comparator chain, a small finite-state machine that visits one node per cycle, or a lookup table when the feature space is aggressively quantized.

For a tree with N_{nodes} internal nodes and N_{leaves} leaves, an approximate storage model is:

$$\begin{aligned}\text{Storage} \approx N_{\text{nodes}} \times (B_{\text{feature}} + B_{\text{threshold}} + B_{\text{child}}) \\ + N_{\text{leaves}} \times B_{\text{phase}} + B_{\text{per-core-state}}.\end{aligned} \tag{2}$$

The latency of a simple comparator-chain or one-node-per-cycle FSM is:

$$\text{Latency} \approx \text{tree depth comparator stages}. \tag{3}$$

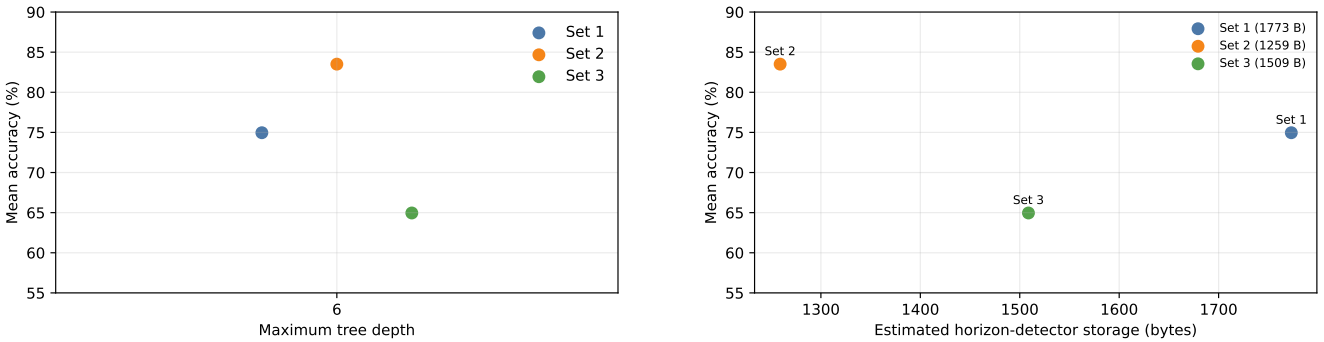
The final horizon detectors use depth-6 trees in all three sets. For the plotted storage estimate, we use the repository's learned node counts together with a compact feature-id, threshold, child-pointer, and phase-code model for the five-tree bank. This gives 100 possible history features, requiring seven feature-id bits. Applying the repository's storage model to the learned tree banks yields 1.26–1.77 KB across the five future-step trees, depending on the set. These are model-storage estimates, not synthesized RTL area or power.

6.2 Per-Core State

Each core stores the most recent selected-counter history for the 20-interval window plus any action-side bookkeeping. With five monitored counters, the online state is still dominated by the history buffer and the tree representation rather than by counter multiplicity. The hardware-visible cost therefore scales primarily with the number of monitored counters, the history length, and the total number of tree nodes.

6.3 Counter-Family Implementation

The cache and LLC family uses events such as L1 data loads/stores, LLC references, and LLC misses. The memory/off-core family uses off-core demand reads and memory bandwidth counters when uncore access is available. The branch family uses branch instructions and branch mispredictions. The instruction/floating-point family uses instructions retired and, when available, floating-point arithmetic events. The key hardware constraint is that programmable PMU slots are limited, so a one-per-family selection is more practical than continuously monitoring a broad all-counter set.



(a) Accuracy versus maximum decision-tree depth. Depth is also the comparator-stage upper bound for a serial hardware traversal.

(b) Estimated tree-bank storage versus accuracy. Storage is an upper bound derived from the learned five-tree horizon detector.

Figure 11: Tree-depth and hardware-cost views of the same detector points. The depth panel exposes the comparator-stage upper bound; the storage panel converts the learned tree banks into the model-storage estimate used in Section 6.1.

6.4 Action Lookup

The detector output can index a compact phase-to-action table similar in spirit to OS policy lookup tables, but closer to the hardware control loop. For example, a state tuple across families may map to a frequency request, cache policy, prefetch level, or scheduling hint. Unlike a software table, the hardware table must be small, deterministic, and protected against rapid oscillation. The hysteresis filter provides this protection by withholding action changes until a state persists.

6.5 Expected Savings and Limits

The expected detector-level savings are fewer monitored counters, lower inference cost than neural sequence models, and less reconfiguration noise than a raw threshold policy. System-level energy or performance savings are not claimed here because the artifacts evaluate detection quality rather than applying a real action policy. This distinction matters: a low-overhead detector can still hurt performance if its action table is poorly tuned, and an accurate detector can still be unusable if it triggers too many hardware changes.

7 Conclusion and Future Directions

This paper presents a lightweight phase-detection flow for multicore workloads based on correlation-guided counter reduction, counter-family ablations, train-only clustered phase labels, and long-horizon decision-tree forecasting over short selected-counter histories. The goal is a detector that is accurate enough to guide hardware adaptation while remaining simple enough to implement with counters, comparators, small tables, and bounded per-core state. The results support the main feasibility story: offline clustering over the full safe counter set can define interpretable low/moderate/high pressure states, ablation can reduce the online interface to five counters, and a bank of shallow trees can forecast the next five phase labels with useful fidelity and clear gains over simple persistence baselines.

The main limitations are equally clear. The current artifacts do not include matched all-counter, GMM, or neural student baselines on the same splits, and they do not connect predicted phase horizons to real DVFS, cache, prefetch, or scheduling actions. Future work should therefore include a direct comparison against richer online students, an RTL prototype, action-level evaluation using the five-step horizon, SPEC CPU2017 expansion, online adaptive trees, cross-architecture validation, and extension to heterogeneous CPU/GPU systems.

References

[1] David H. Albonesi. Selective cache ways: On-demand cache resource allocation. In *Proceedings of the 32nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 248–259, 1999.

[2] Rajeev Balasubramonian, David H. Albonesi, Alper Buyuktosunoglu, and Sandhya Dwarkadas. Memory hierarchy reconfiguration for energy and performance in general-purpose processor architectures. In *Proceedings of the 33rd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 245–257, 2000.

[3] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. The parsec benchmark suite: Characterization and architectural implications. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*, pages 72–81, 2008.

[4] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[5] Alper Buyuktosunoglu, Stanley Schuster, David Brooks, Pradip Bose, Peter Cook, and David Albonesi. An adaptive issue queue for reduced power at high performance. In *Power-Aware Computer Systems*, pages 25–39, 2001.

[6] Lorenzo Carpentieri, Antonio De Caro, Majid Salimibeni, Kaijie Fan, and Biagio Cosenza. Phase-based frequency scaling for energy-efficient heterogeneous computing. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, pages 824–836, 2025.

[7] Chih-Hao Chang, Pangfeng Liu, and Jan-Jan Wu. Sampling-based phase classification and prediction for multi-threaded program execution on multi-core architectures. In *Proceedings of the 42nd International Conference on Parallel Processing*, pages 349–358, 2013.

[8] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1):1–38, 1977.

[9] Gaurav Dhiman, Giovanni Marchetti, and Tajana Rosing. vgreen: A system for energy-efficient computing in virtualized environments. In *Proceedings of the International Symposium on Low Power Electronics and Design*, pages 243–248, 2009.

[10] Ashutosh S. Dhodapkar and James E. Smith. Managing multi-configuration hardware via dynamic working set analysis. In *Proceedings of the 29th International Symposium on Computer Architecture*, pages 233–244, 2002.

[11] Ashutosh S. Dhodapkar and James E. Smith. Comparing program phase detection techniques. In *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 217–227, 2003.

[12] Chen Ding, Sandhya Dwarkadas, Michael C. Huang, Kai Shen, and John B. Carter. Program phase detection and exploitation. In *Proceedings of the 20th IEEE International Parallel and Distributed Processing Symposium*, 2006.

[13] Stijn Eyerman, Lieven Eeckhout, Tejas Karkhanis, and James E. Smith. A performance counter architecture for computing accurate cpi components. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 175–184, 2006.

[14] Greg Hamerly, Erez Perelman, Jeremy Lau, and Brad Calder. Simpoint 3.0: Faster and more flexible program phase analysis. *Journal of Instruction-Level Parallelism*, 7:1–28, 2005.

[15] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition, 2019.

[16] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[17] Intel Corporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 2024.

[18] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*, 2026.

[19] Canturk Isci and Margaret Martonosi. Runtime power monitoring in high-end processors: Methodology and empirical data. In *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 93–104, 2003.

[20] Anil K. Jain, M. Narasimha Murty, and Patrick J. Flynn. Data clustering: A review. *ACM Computing Surveys*, 31(3):264–323, 1999.

[21] Yeseong Kim, Pietro Mercati, Ankit More, Emily Shriver, and Tajana Rosing. p^4: Phase-based power/performance prediction of heterogeneous systems via neural networks. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, pages 683–690, 2017.

[22] Jeremy Lau, Stefan Schoenmackers, Timothy Sherwood, and Brad Calder. Structures for phase classification. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software*, 2004.

[23] Linux Kernel Developers. *perf_event_open(2) Linux Programmer's Manual*, 2024.

[24] Linux perf developers. perf: Linux profiling with performance counters. `https://perfwiki.github.io/main/`, 2024.

[25] Stuart P. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[26] Erika Susana Alcorta Lozano and Andreas Gerstlauer. Learning-based phase-aware multi-core cpu workload forecasting. *ACM Transactions on Design Automation of Electronic Systems*, 28(2), 2022.

[27] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, pages 281–297, 1967.

[28] Paul E. McKenney and John D. Slingwine. Read-copy update: Using execution history to solve concurrency problems. In *Proceedings of the 10th International Parallel Processing Symposium*, 1995.

[29] Andreas Merkel and Frank Bellosa. Balancing power consumption in multiprocessor systems. In *Proceedings of the 1st ACM SIGOPS/EuroSys European Conference on Computer Systems*, pages 403–414, 2006.

[30] Junaid Nomani and Jakub Szefer. Predicting program phases and defending against side-channel attacks using hardware performance counters. In *Proceedings of the Fourth Workshop on Hardware and Architectural Support for Security and Privacy*, 2015.

[31] Erez Perelman, Marzia Polito, Jean-Yves Bouguet, John Sampson, Brad Calder, and Carole Dulong. Detecting phases in parallel applications on shared memory architectures. In *Proceedings of the 20th IEEE International Parallel and Distributed Processing Symposium*, 2006.

[32] Vinicius Petrucci, Orlando Loques, and Daniel Mosse. Lucky scheduling for energy-efficient heterogeneous multi-core systems. In *Proceedings of the USENIX Workshop on Hot Topics in Power-Aware Computing and Systems*, 2012.

[33] J. Ross Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.

[34] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[35] Timothy Sherwood, Erez Perelman, and Brad Calder. Basic block distribution analysis to find periodic behavior and simulation points in applications. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, pages 3–14, 2001.

[36] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. Automatically characterizing large scale program behavior. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 45–57, 2002.

[37] Timothy Sherwood, Suleyman Sair, and Brad Calder. Phase tracking and prediction. In *Proceedings of the 30th International Symposium on Computer Architecture*, pages 336–349, 2003.

[38] Weihua Zhang, Jiaxin Li, Yi Li, and Haibo Chen. Multilevel phase analysis. *ACM Transactions on Embedded Computing Systems*, 14(2):31:1–31:29, 2015.

[39] Ibrahim E. Ziedan, Hazem I. Shehata, and Shaymaa M. Serag. A run-time program phase detection technique for optimizing per-phase l2 cache demand. *The Egyptian International Journal of Engineering Sciences and Technology*, 20:1–9, 2016.